

AI Interview Report

Role: Python Developer

Difficulty: Easy | Interviewer style: Friendly

Date: 2026-09-01

Overall score: 7.5 / 10

Filler words used: 1

Summary

The candidate demonstrated solid Python fundamentals, showcasing their ability to design and implement data structures, write clean code, and optimize functions. Their knowledge of decorators and testing frameworks was also impressive. However, the candidate could benefit from improving their performance optimization techniques and code efficiency, particularly in handling large datasets. Overall, this candidate would be a strong fit for a junior Python developer role.

Q1 [TECHNICAL] Score: 9/10

What is the difference between a Python list and a tuple, and when would you use one over the other?

Your answer:

Lists are mutable and intended for dynamic data that changes, whereas tuples are immutable, memory-efficient, and used for fixed records or hashable dictionary keys.

Strengths:

- + Clear distinction between list and tuple
- + Simple explanation of key characteristics

Suggestions:

- Consider adding a concrete example or edge case to illustrate the practical application of this distinction
- You may want to briefly mention the implications of immutability on data sharing and modification

Improved answer:

Lists are mutable and intended for dynamic data that changes, whereas tuples are immutable, memory-efficient, and used for fixed records or hashable dictionary keys. For instance, using a list to represent a collection of student grades might require frequent updates, whereas using a tuple to store a fixed set of exam scores would be more suitable.

Q2 [BEHAVIORAL] Score: 8/10

Can you explain how you would test the effectiveness of a decorator, and how would you ensure that it is properly applied to a function in your testing framework of choice?

Your answer:

Using pytest, you can test a decorator's effectiveness by verifying its side effects (like logs or execution time) using caplog or mocks, and ensure proper application by asserting that the function preserves its original metadata via functools.wraps.

Strengths:

- + Provides a good overview of testing decorators with pytest

Suggestions:

- Consider providing more specific examples or edge cases to demonstrate the testing approach

AI Interview Report

Improved answer:

When testing the effectiveness of a decorator with pytest, you can leverage caplog to verify its side effects, such as logging statements or execution time metrics. Additionally, you can use pytest-mock to isolate dependencies and test the decorator's behavior under various scenarios. To ensure proper application, assert that the decorated function preserves its original metadata using functools.wraps, and also verify that the decorator's metadata is correctly propagated to the decorated function. For instance, you can test the decorator's docstring and module-level docstring to ensure they are preserved correctly.

Filler words: 1

Q3 [TECHNICAL] Score: 6/10

Write a simple Python class that uses a generator to yield a sequence of numbers from 1 to 10, and then use a list comprehension to create a new list that doubles each number in the sequence, without using any explicit loops or for loops.

Your answer:

```
class NumberSequence:
```

```
    def numbers(self):
```

```
        n = 1
```

```
        while n <= 10:
```

```
            yield n
```

```
            n += 1
```

```
# Usage without explicit for loops in the generator logic:
```

```
seq = NumberSequence()
```

```
doubled_numbers = [x * 2 for x in seq.numbers()]
```

```
print(doubled_numbers)
```

Strengths:

- + Created a generator to yield a sequence of numbers

Suggestions:

- Consider using a more Pythonic way to define the sequence, such as using a range function.
- Avoid using a class with a method name that is a verb (numbers) and instead use a more descriptive name, such as Sequence or NumberGenerator.

Improved answer:

```
class NumberSequence:
```

```
    def __iter__(self):
```

```
        n = 1
```

```
        while n <= 10:
```

```
            yield n
```

```
            n += 1
```

```
# Usage without explicit for loops in the generator logic:
```

AI Interview Report

```
seq = NumberSequence()
doubled_numbers = [x * 2 for x in seq]
```

Q4 [PROBLEM-SOLVING] Score: 7/10

Write a Python function that takes a list of integers as input and returns a dictionary where the keys are the unique integers in the list and the values are the counts of each integer. Use the `functools` module to make the function more efficient. Use a generator expression to calculate the counts instead of a traditional for loop.

Your answer:

```
import functools
```

```
def count_integers(numbers):
    return functools.reduce(
        lambda counts, x: counts.update({x: counts.get(x, 0) + 1}) or counts,
        (n for n in numbers),
        {},
    )
```

Strengths:

- + uses `functools.reduce` for efficiency
- + uses a generator expression

Suggestions:

- the `update` method modifies the dictionary in-place, which can lead to unexpected side effects; consider using a temporary dictionary instead
- the return value of the lambda function is not used; consider adding a return statement to make the code more explicit

Improved answer:

```
import functools
def count_integers(numbers):
    counts = {}
    return functools.reduce(
        lambda counts, x: (counts[x] := counts.get(x, 0) + 1) or counts, (
            n for n in numbers
        ), counts
    )
```